1 Introduction and Overview
2 Data Wrangling
3 Data Exploration
4 Feature Engineering
5 Image Captioning
6 Multimodal Sentiment Analysis
7 Visual Question Answering
8 Text to Image Generation
9 Text to Video Generation

Unlock Full Access with 30% off

Login

01
Introduction and Overview

This book is designed to help machine learning (ML) engineers and data scientists succeed in ML system design interviews, with a focus on generative AI systems. We begin with a brief overview of generative AI, then discuss the ML system design process (simplified and in a fundamental manner), such as search and recommendation systems. This book, however, explores Gen AI applications and the ML system design process for Gen AI applications. This chapter sets the stage for the rest of the book, and a guide for those who want to prepare for a Gen AI related interview.

This chapter explores two key topics. First, it provides an overview of Gen AI, delving into its fundamental concepts and applications. Then, it introduces a comprehensive framework for ML system design interviews. This is essential for real-world applications and interview preparation. This framework will serve as the foundation for developing popular Gen AI systems in the subsequent chapters.

GenAI overview

What are AI and ML?

AI is a broad yet popular course focused on creating systems that can perform tasks typically requires human intelligence, such as reasoning, planning, and problem-solving. As a subset of AI that gains significance in recent years, machine learning (ML) enables these systems to learn from data and improve their performance over time. ML encompasses various techniques, including supervised learning, unsupervised learning, and reinforcement learning, each with distinct methods and applications. Deep learning, a specialized subset of ML, utilizes neural networks to handle complex tasks, such as image recognition and natural language processing.

Figure 1: The relationship between AI and ML

Discriminative

Discriminative models classify data by learning the differences between classes based on input features. Formally, they learn the conditional probability, $P(Y|X)$, where Y represents the target variable and X represents the input features.

Discriminative models are commonly used in various applications where the goal is to categorize data into one of several classes, such as spam detection, sentiment analysis, or image recognition. For instance, in a binary classification task, a discriminative model might predict whether an email is spam or not spam based on features like the sender's email address, the subject line, and the content of the email. The model would learn to distinguish between these features to accurately classify new emails as spam or not spam.

Common algorithms for developing discriminative models include:

- **Logistic regression:** Despite its name, it is a classification algorithm that predicts the probability of various outcomes based on input features.
- **Support vector machines (SVMs):** SVMs find the hyperplane that best separates different classes in the feature space. They can be effective in high-dimensional spaces and for complex decision boundaries.
- **Decision trees:** These models make decisions by asking a series of questions about the input features, with each question narrowing down the target variable.
- **Neural networks (NNs):** These powerful, flexible models can learn complex patterns and relationships in data, making them suitable for various tasks, including image recognition and natural language processing.

Generative models

Generative models aim to understand and replicate the underlying distribution of data. Formally, they model the distribution $P(X)$ where X represents the training data. By capturing this distribution, generative models can create new data points that resemble the input data and the target variable, Y, from Bayes' perspective. This allows them to generate new data instances by sampling from the learned distribution.

Generative models encompass various techniques, each with its distinguishing traits and applications. Some prominent approaches include:

- **Generative adversarial networks (GANs):** GANs pit two neural networks against each other in a competitive manner. The generator creates fake data, and the discriminator tries to distinguish between real and generated data. This adversarial process results in the generator producing increasingly realistic outputs.
- **Diffusion models:** These models gradually add noise to data examples until they become pure noise, then learn to reverse this process. They can generate high-quality images and other data types.
- **Autoregressive models:** These models generate data by predicting each element in a sequence based on the preceding elements. They are commonly used for text generation and time series forecasting.

Discriminative and generative models are used for different purposes. Discriminative models are used for tasks where the goal is to classify or predict, generative models are used to generate new examples. Figure 2 shows popular tasks pertaining to generative and discriminative models.

Figure 2: Popular AI-powered tasks

What is GenAI and why is it gaining popularity?

Gen AI models can create various generative algorithms to learn and operate capabilities across data samples such as images, videos, text, and audio.

Gen AI has gained a lot of popularity recently for two main reasons. First, these models can perform various tasks across different domains, such as generating text, images, and multimedia content. This versatile nature allows businesses to leverage these capabilities across industries, from creative arts and entertainment to healthcare and software development.

Second, these models can significantly boost productivity by automating tasks that traditionally require human creativity and generate vast quantities of high-quality content in a fraction of the time it would take a human. This increased efficiency can lead to cost savings, faster time-to-market for new products and services, and the ability to scale content creation efforts to meet the demands of the digital age. For instance, a marketing team can use Gen AI to create blog posts, social media updates, and ad copy much faster than a human team, allowing them to focus on other strategic initiatives.

Figure 3: Gen AI models and applications

Because of easy access to no-code/low-code tools that democratize the creation process, even non-technical users can create sophisticated outputs. This democratization fosters innovation across various sectors, empowering individuals to leverage AI capabilities for their specific needs and goals. The ease of use and the potential for high-quality results have spurred widespread adoption and experimentation with Gen AI technologies, further accelerating its growth and impact on various industries.

How Gen AI is becoming so powerful?

Gen AI models have recently demonstrated impressive capability and are becoming very powerful. Three key factors driving this progress are:

- Scaling laws
- Transfer capacity
- Emergent abilities

Scaling laws

Gen AI models's effectiveness depends on its training data. For example, a 7-billion-parameter model has been trained on extensive medical texts, it might struggle to diagnose diseases accurately. Improving a model for a specific task requires large datasets with labels, but collecting such data can be expensive and time-consuming.

Scaling laws offer a solution by suggesting that increasing model size, training data, and computing power can lead to better performance across various tasks. Researchers have found that even simple models can achieve impressive results when trained on large datasets. These scaling laws have fueled the development of larger and more powerful AI models, as they suggest that investing in more data and computing power can yield significant improvements in model capabilities.

Transfer capacity

An notable increase in capacity, their performance tends to improve, task-specific training data large models requires enormous amounts of computational resources. The computer required during model training is often measured in FLOP, representing the total number of floating-point operations a computer can perform per second while running a specific program. Training a large language model with billions of parameters, such as those in the GPT series, can demand hundreds or even thousands of FLOP.

To manage these resources, companies often utilize specialized cloud computing services from providers like Amazon Web Services (AWS), Google Cloud Platform (GCP), or Microsoft Azure. These cloud platforms offer access to powerful hardware, including graphics processing units (GPUs) and tensor processing units (TPUs), optimized for machine learning tasks. The flexibility of cloud computing allows companies to scale their resources up or down as needed, optimizing cost-efficiency while ensuring they have the necessary computing power for their AI projects.

Emergent abilities

Scaling laws indicate that larger models with more parameters and trained on more data tend to exhibit better performance across various tasks. However, some advanced capabilities do not gradually improve with model size but instead emerge suddenly in models that have reached a certain threshold of scale. These emergent abilities include:

- **Chain of thought reasoning:** This allows models to break complex problems into a series of logical steps, mimicking human-like reasoning processes.
- **Instruction following:** Larger models demonstrate a more nuanced understanding of and adherence to complex instructions, enabling more precise control over their outputs.
- **Multi-step problem solving:** These models can tackle problems requiring multiple steps or stages of reasoning, showcasing a form of planning and execution.

These emergent abilities represent qualitative leaps in model capabilities that weren't predicted by simple extrapolation of performance metrics from smaller models. The sudden nature of these emergent abilities has prompted researchers to study them closely, as they can significantly impact the model's overall utility and performance across various tasks.

Why Gen AI is becoming so powerful?

Gen AI models's effectiveness depends on its training data. For example, a 7-billion-parameter model has been trained on extensive medical texts, it might struggle to diagnose diseases accurately. Improving a model for a specific task requires large datasets with labels, but collecting such data can be expensive and time-consuming.

Scaling laws offer a solution by suggesting that increasing model size, training data, and computing power can lead to better performance across various tasks. Researchers have found that even simple models can achieve impressive results when trained on large datasets. These scaling laws have fueled the development of larger and more powerful AI models, as they suggest that investing in more data and computing power can yield significant improvements in model capabilities.

Model capacity

Another key factor in the effectiveness of ML models is their capacity to learn. Model capacity is measured in two ways:

- Number of parameters
- FLOP count

Number of parameters

Parameters are the adjustable elements within a model that are learned during the training process. The number of parameters is a key indicator of a model's capacity to learn from data.

A simple model with only a few parameters might struggle to capture the complex patterns and relationships that exist within large datasets. However, adding more parameters allows the model to learn more intricate details and nuances in a large dataset. Table 1 shows the popular models and their number of parameters.

Model Name	Parameters
Google's PaLM [1]	540B
Deepmind's GPT-2 [2]	1.5B
Google's Flamingo [3]	80B
Meta's Llama 3 [4]	405B
Google's Imagen [5]	2B

Table 1: Popular Gen AI models and their number of parameters

FLOP count

The number of floating-point operations (FLOP) required to complete a computation serves as a proxy for the complexity of the model's architecture. More complex models with more parameters typically require more FLOP for training and inference. For instance, a deep learning model with many layers and numerous connections between neurons will need more FLOP to process data and make predictions compared to a simpler model with fewer layers and connections.

Figure 4: Scaling laws for GPT-2 models

Scaling law

While the importance of parameters is clear, the relationship between model size and performance is not straightforward. Research on large language models (LLMs) has shown that increasing model size can lead to improvements in various tasks, but the relationship is not linear. The performance gains may plateau or even decrease beyond a certain point, depending on factors such as the quality and quantity of the training data and the specific architecture of the model.

Scaling laws provide a framework for understanding how model performance changes with size. These laws suggest that there is a power-law relationship between model size and performance, meaning that doubling the model size might lead to a specific percentage increase in performance metrics. However, this relationship is not universal and may vary across different tasks and domains.

Figure 4 illustrates the scaling laws for GPT-2 models, showing how cross-entropy loss (a measure of model performance) decreases with increasing model size, dataset size, and compute power. The graph shows that larger models tend to perform better across these factors, but the rate of improvement varies for each factor.

Transfer capacity

Transfer capacity refers to a model's ability to leverage knowledge learned from one task to improve performance on other related tasks. This capability is crucial for developing versatile and efficient AI systems that can adapt to various applications without requiring extensive retraining for each new task.

Transfer capacity allows models to generalize their learning across different domains and tasks, potentially reducing the amount of data and computational resources needed for training on new tasks. This can be particularly beneficial when dealing with limited data for specific tasks or when quickly adapting to new, emerging tasks.

Large language models (LLMs) have demonstrated impressive transfer capacity, showing the ability to perform various tasks with minimal or no additional training. For instance, a single LLM might be capable of translating languages, answering questions, summarizing text, and even writing simple computer programs, all without being explicitly trained for each specific task.

Emergent abilities

Emergent abilities refer to capabilities that suddenly appear in large language models (LLMs) as they reach a certain threshold of scale, rather than gradually improving with model size. These abilities are not predicted by simple extrapolation of performance metrics from smaller models and can significantly impact the model's overall utility and performance across various tasks.

Some notable emergent abilities in LLMs include:

- **Chain of thought reasoning:** This allows models to break complex problems into a series of logical steps, mimicking human-like reasoning processes.
- **Instruction following:** Larger models demonstrate a more nuanced understanding of and adherence to complex instructions, enabling more precise control over their outputs.
- **Multi-step problem solving:** These models can tackle problems requiring multiple steps or stages of reasoning, showcasing a form of planning and execution.

These emergent abilities represent qualitative leaps in model capabilities that weren't predicted by simple extrapolation of performance metrics from smaller models. The sudden nature of these emergent abilities has prompted researchers to study them closely, as they can significantly impact the model's overall utility and performance across various tasks.

A Framework for ML system design interviews

Many engineers think of ML algorithms first when tackling ML system design interviews. However, this approach can lead to inefficient and ineffective solutions. A more comprehensive approach involves considering various factors beyond the algorithm, including data quality, scalability, and computational resources. This holistic approach ensures that the proposed solution addresses the problem at hand and leverages the most appropriate tools and techniques for optimal performance.

To succeed in these interviews, candidates need to demonstrate a structured approach to problem-solving. This involves asking clarifying questions to understand the problem fully, considering various trade-offs between different approaches, and justifying decisions based on the specific requirements and constraints of the problem. By following a structured framework, candidates can demonstrate their ability to think critically about complex ML system design problems and propose well-reasoned solutions that balance various factors and considerations.

Clarifying Requirements

When you start to develop an ML system to solve a particular task, you often have minimal information to begin with. Similarly, in interviews, you might be presented with a vague prompt like "Design a spam detection system" or "Design a news feed for your friends." These prompts lack specific details about the system's requirements, leaving many aspects open to interpretation.

Your questions should help in understanding the problem space and the specific goals the system needs to achieve. They fall into two main categories:

- Functional requirements
- Non-functional requirements

Functional requirements

Functional requirements clarify what the system should do from the user's perspective. For example, questions like "Should we allow users to tag friends in their posts?" or "Should we provide feedback on the spam we detect?" help define the system's core functionalities and user interactions. These requirements directly impact the system's design and implementation.

Non-functional requirements

Non-functional requirements focus on how the system performs, not what it does. These include aspects like scalability, latency, and security. For instance, questions like "How many users are expected to use the system?" or "How quickly should the system respond to requests?" help define the system's performance characteristics and resource requirements. These factors significantly influence the choice of algorithms, data structures, and infrastructure used to build the system.

Framing the problem as an ML task

After a clear understanding of the problem and its requirements, the next step involves translating these business requirements into machine learning tasks. This crucial step helps in defining the problem in a way that machine learning algorithms can understand and address effectively.

When framing a problem, it's important to consider various machine learning tasks such as classification, regression, or clustering. The chosen task will significantly impact the overall approach to solving the problem. For example, if the goal is to detect spam, we might frame it as a binary classification task, where the system classifies each incoming message as either spam or not spam.

Choose a suitable ML approach

After defining the input and output of the system, the next step is to choose the most suitable ML approach. This involves considering various factors, including the complexity of the problem, the available data, and the desired performance metrics. The following steps can help you narrow down your options when choosing the most suitable approach:

Figure 6: Choosing an algorithm

There are different ways to select an appropriate ML algorithm, and the criteria for selection vary from application to application. The following steps can help you narrow down your options when choosing the most suitable algorithm:

- **Discriminative or generative:** Determine whether the problem requires a discriminative or generative approach. This can guide the selection process, as some algorithms are more suited for discriminative tasks, while others are better for generative tasks. For instance, if the goal is to classify data into different categories, a discriminative model might be a more appropriate choice.
- **Data characteristics:** Consider the nature of the data, including its structure, complexity, and the presence of any inherent biases or patterns. These factors can influence the performance of different algorithms. For example, if the data exhibits complex, non-linear relationships, more sophisticated algorithms like neural networks might be required.
- **Interpretability:** Assess the importance of interpretability in the context of the problem. Some algorithms, such as decision trees or linear models, are more interpretable than others, such as deep neural networks. If interpretability is crucial, it may influence the choice of algorithm.
- **Computational resources:** Evaluate the available computational resources, including processing power, memory, and storage capacity. Some algorithms require more resources than others, and the chosen algorithm should be feasible to implement given the available resources.
- **Performance metrics:** Consider the performance metrics relevant to the problem, such as accuracy, precision, recall, or mean squared error. The chosen algorithm should be capable of achieving acceptable performance on these metrics.

Specify the system's input and output

A clear definition of the system's input and output is crucial for successful ML system design. This involves identifying the input data modality (text, image, audio, video) and the expected output. For instance, in a chatbot system, the input is the user's query, and the output is the system's response.

Figure 7: Input and output of a chatbot

Choose a suitable ML approach

After defining the input and output of the system, the next step is to choose the most suitable ML approach. This involves considering various factors, including the complexity of the problem, the available data, and the desired performance metrics. The following steps can help you narrow down your options when choosing the most suitable approach:

Figure 8: Choosing an algorithm

There are different ways to select an appropriate ML algorithm, and the criteria for selection vary from application to application. The following steps can help you narrow down your options when choosing the most suitable algorithm:

- **Discriminative or generative:** Determine whether the problem requires a discriminative or generative approach. This can guide the selection process, as some algorithms are more suited for discriminative tasks, while others are better for generative tasks. For instance, if the goal is to classify data into different categories, a discriminative model might be a more appropriate choice.
- **Data characteristics:** Consider the nature of the data, including its structure, complexity, and the presence of any inherent biases or patterns. These factors can influence the performance of different algorithms. For example, if the data exhibits complex, non-linear relationships, more sophisticated algorithms like neural networks might be required.
- **Interpretability:** Assess the importance of interpretability in the context of the problem. Some algorithms, such as decision trees or linear models, are more interpretable than others, such as deep neural networks. If interpretability is crucial, it may influence the choice of algorithm.
- **Computational resources:** Evaluate the available computational resources, including processing power, memory, and storage capacity. Some algorithms require more resources than others, and the chosen algorithm should be feasible to implement given the available resources.
- **Performance metrics:** Consider the performance metrics relevant to the problem, such as accuracy, precision, recall, or mean squared error. The chosen algorithm should be capable of achieving acceptable performance on these metrics.

This list isn't comprehensive, but it provides a good starting point. Other factors, such as privacy, ethics, and data security, can also influence the decision.

By the end of this step, you should be clear about the system's input and output and have a suitable ML approach in mind. The next step involves designing the system's overall architecture and data flow.

Design the system's overall architecture and data flow

Designing the system's overall architecture and data flow is crucial for ensuring efficiency, scalability, and maintainability. This step involves breaking down the problem into smaller, manageable components and defining how data will flow between these components. The following steps can help you design a robust and efficient system architecture:

- Data collection
- Data preprocessing
- Feature extraction
- Model training
- Model evaluation
- Model serving
- Monitoring
- Feedback collection

Figure 9: Data collection to feedback collection

- **Identify the main tasks:** Start by identifying the specific main tasks or functions within the system's architecture. The main components might include data ingestion, preprocessing, feature extraction, model training, evaluation, serving, monitoring, and feedback collection. The system's output can vary depending on the task at hand. For instance, in a spam detection system, the output might be a binary classification (spam or not spam). However, in a more complex system, the output could be a ranked list of relevant items or a personalized feed, as in a news feed system.
- **Choose a suitable algorithm:** Finally, select an algorithm that is most suitable based on the requirements. Consider factors such as the complexity of the problem, the available data, and the desired performance metrics. For instance, if the problem involves processing large amounts of data in real-time, a simple and lightweight algorithm might be preferred to ensure low latency and high throughput. Conversely, if the problem involves complex patterns and relationships in the data, a more sophisticated algorithm like a deep neural network might be necessary to achieve the desired performance.
- **Define how data flows between these components:** Next, focus on how data will flow between these components. This involves defining the interfaces and protocols for data exchange between different components, ensuring that the system can handle the volume and velocity of data required for its operation.
- **Consider scalability and fault tolerance:** Think about how the system can be scaled to handle increasing data volumes and user traffic. Consider techniques such as load balancing, caching, and distributed processing to ensure the system can maintain performance and reliability as it grows.

Data collection

Data is the lifeblood of any machine learning (ML) system, and the quality and quantity of the data collected can significantly impact the performance of the ML model. The following steps can help you gather the necessary data for your ML system:

- Identify data sources
- Collect data
- Ensure data quality
- Address ethical and legal considerations

Figure 10: Data collection process

- **Identify data sources:** Start by identifying the potential sources of data that can be used to train the ML model. These sources might include internal databases, third-party datasets, or even data collected from sensors or user interactions. The choice of data sources will depend on the specific task and the type of data required.
- **Collect data:** Once the data sources have been identified, the next step is to collect the data from these sources. This may involve using APIs to extract data from online platforms, scraping data from websites, or manually collecting data through surveys or interviews. It's important to ensure that the data collection process is compliant with relevant legal and ethical guidelines.
- **Ensure data quality:** The quality of the data is crucial for the performance of the ML model. This step involves cleaning the data to remove any errors, inconsistencies, or missing values. It may also involve data augmentation techniques to increase the diversity and volume of the data.
- **Address ethical and legal considerations:** When collecting data, it's important to consider the ethical and legal implications of data privacy and ownership. This may involve obtaining consent from individuals whose data is being collected, anonymizing sensitive data, and ensuring that the data is used in a fair and unbiased manner.

Data preprocessing

Raw data is rarely in a suitable format for machine learning models. Data preprocessing is a crucial step that involves cleaning, transforming, and organizing the data to make it suitable for the ML model. The following steps are commonly involved in data preprocessing:

- Handle missing values
- Remove duplicate data
- Normalize or standardize numerical features
- Encode categorical variables
- Tokenize and stem text data
- Resize and normalize images
- Extract relevant features from the data

Figure 11: Data preprocessing steps

Structured data: This type of data can be organized into tables with rows and columns. For example, a database or spreadsheet containing financial records with various attributes like transaction amount, date, and category would be considered structured data.

- **Categorical data:** Data that represents distinct groups or categories, e.g., gender or publication source.
- **Numerical data:** Data that represents quantities or measurements, e.g., transaction amount or age.

Unstructured data: Unstructured data does not adhere to a predefined data model or organizational structure. Examples include text documents, social media posts, or even sensor readings from IoT devices.

- **Text data:** This data can be in various formats, including plain text, HTML, or JSON. Preprocessing data from various sources may involve parsing and extracting relevant information from these different formats.
- **Image data:** Image data can be in various formats, such as JPEG or PNG. Preprocessing image data may involve resizing, normalizing pixel values, or extracting features using techniques like edge detection or color histograms.

Time-dependent data: This type of data is collected over time and can be structured or unstructured. It is important to consider the temporal aspect when preprocessing this data, as it may involve techniques like time series analysis or feature extraction that capture temporal patterns.

- **Sensor data:** Data from sensors can be in various formats, depending on the specific sensor. Preprocessing sensor data may involve techniques like noise reduction, data imputation, or feature extraction to capture relevant information.
- **Log data:** Log data is typically in a semi-structured format, such as JSON or plain text. Preprocessing log data may involve parsing and extracting relevant information, such as timestamps, user IDs, or error codes.

Feature engineering

Feature engineering involves selecting and extracting predictive features from raw data and transforming them into a format suitable for the model. This process often requires domain knowledge and can significantly impact the model's performance. The following steps are commonly performed in feature engineering and serving features at scale:

- Feature selection
- Feature extraction
- Feature transformation
- Feature scaling
- Feature storage and serving

Figure 12: Feature engineering steps

- **Feature selection:** This involves choosing the most relevant features from the available data. It's important to choose features that provide the most information about the target variable while avoiding redundant or irrelevant features. Techniques such as feature ranking, dimensionality reduction, or using domain knowledge can be employed to select the most informative features.
- **Feature extraction:** Sometimes, the raw data may not be in a suitable format for the model. Feature extraction involves transforming the raw data into a more suitable format by extracting relevant information. For example, in text data, techniques like tokenization, stemming, or lemmatization can be used to extract meaningful features from the text.
- **Feature transformation:** This step involves transforming the features to improve their suitability for the model. Common transformations include normalizing numerical features to a standard scale, encoding categorical variables into numerical representations, or creating interaction terms between features.
- **Feature scaling:** Many machine learning algorithms are sensitive to the scale of the features. Feature scaling ensures that all features are on a similar scale, preventing some features from dominating the model's learning process. Techniques like min-max scaling or standardization can be used to scale the features.
- **Feature storage and serving:** Once the features have been engineered, they need to be stored and served to the model during training and inference. This may involve using a feature store or a database optimized for storing and retrieving features at scale.

Model training

Model training involves feeding the prepared data into the chosen algorithm to learn the patterns and relationships between the input features and the target variable. The following steps are commonly involved in model training:

- Data splitting
- Hyperparameter tuning
- Cross-validation
- Regularization
- Ensemble methods
- Transfer learning
- Fine-tuning
- Prompt engineering
- Retrieval augmented generation
- Chain of thought reasoning
- Multi-agent systems
- Human-in-the-loop
- Explainable AI
- Ethical AI
- Bias mitigation
- Fairness assessment
- Privacy preserving techniques
- Federated learning
- Differential privacy
- Synthetic data generation
- Data augmentation
- Feature engineering
- Hyperparameter tuning
- Model compression
- Quantization
- Pruning
- Distillation
- Ensemble methods
- Transfer learning
- Fine-tuning
- Prompt engineering
- Retrieval augmented generation
- Chain of thought reasoning
- Multi-agent systems
- Human-in-the-loop

Figure 13: Model training steps

- **Data splitting:** The data is divided into training, validation, and test sets. The training set is used to train the model, the validation set is used to tune hyperparameters and evaluate the model's performance during training, and the test set is used to evaluate the final performance of the model on unseen data.
- **Hyperparameter tuning:** Hyperparameters are settings that control the learning process, such as the learning rate or the number of layers in a neural network. Tuning these hyperparameters can significantly impact the model's performance.
- **Cross-validation:** This technique involves dividing the training data into multiple folds and training the model on different combinations of these folds. It helps ensure the model's robustness and generalizability.
- **Regularization:** Regularization techniques like L1 or L2 regularization can prevent overfitting by adding a penalty to the model's loss function based on the complexity of the model.
- **Ensemble methods:** Ensemble methods combine multiple models to improve performance. Techniques like bagging, boosting, or stacking can be used to create a more robust and accurate model.
- **Transfer learning:** Transfer learning involves using a pre-trained model as a starting point and fine-tuning it for the specific task. This can save time and resources while leveraging the knowledge learned by the pre-trained model.
- **Fine-tuning:** Fine-tuning involves adjusting the weights of a pre-trained model on a specific dataset to adapt it to the new task. This can be more efficient than training a model from scratch.
- **Prompt engineering:** For large language models, prompt engineering involves crafting the input prompts to guide the model's behavior and elicit the desired outputs.
- **Retrieval augmented generation (RAG):** RAG combines retrieval and generation models to leverage external knowledge bases and improve the model's performance on tasks requiring factual knowledge.
- **Chain of thought reasoning:** This technique prompts the model to break complex problems into a series of logical steps, mimicking human-like reasoning processes.
- **Multi-agent systems:** These systems involve multiple models interacting with each other to solve complex tasks, simulating a form of collaborative intelligence.
- **Human-in-the-loop:** This approach involves incorporating human feedback and expertise into the model training process, ensuring the model's outputs align with human preferences and values.
- **Explainable AI (XAI):** XAI techniques aim to make the model's decision-making process more transparent and interpretable, which is crucial for building trust and understanding the model's behavior.
- **Ethical AI:** This encompasses various considerations and practices to ensure the model is developed and used responsibly, addressing issues like bias, fairness, and potential societal impacts.
- **Bias mitigation:** These techniques aim to identify and reduce biases in the model's training data and predictions, promoting fairness and equitable outcomes.
- **Fairness assessment:** This involves evaluating the model's performance across different demographic groups to ensure it doesn't exhibit discriminatory behavior.
- **Privacy preserving techniques:** These methods protect sensitive data during the model training process, such as differential privacy or federated learning.
- **Federated learning:** This approach allows training models on decentralized data sources without centralizing the data, enhancing privacy and security.
- **Differential privacy:** This technique adds noise to the data or the model's outputs to prevent the leakage of sensitive information about individual data points.
- **Synthetic data generation:** This involves creating artificial data that mimics the statistical properties of real data, useful for augmenting datasets or addressing privacy concerns.
- **Data augmentation:** This technique involves creating variations of the training data to increase its diversity and volume, helping the model generalize better.
- **Feature engineering:** This involves creating new features from existing data to improve the model's performance.
- **Hyperparameter tuning:** This involves optimizing the model's hyperparameters to achieve the best performance.
- **Model compression:** This techniques reduce the model's size and complexity while maintaining its performance, facilitating deployment on resource-constrained devices.
- **Quantization:** This involves reducing the precision of the model's weights and activations to decrease memory and computational requirements.
- **Pruning:** This technique removes unnecessary connections or neurons in the model to reduce its size and improve efficiency.
- **Distillation:** This involves training a smaller model to mimic the behavior of a larger, more complex model.
- **Ensemble methods:** Combining multiple models to improve overall performance and robustness.
- **Transfer learning:** Leveraging knowledge from pre-trained models on related tasks to improve performance on the target task.
- **Fine-tuning:** Adjusting the weights of a pre-trained model on specific data to adapt it to a new task.
- **Prompt engineering:** Crafting input prompts to guide the model's behavior and elicit desired outputs.
- **Retrieval augmented generation (RAG):** Combining retrieval and generation models to leverage external knowledge bases.
- **Chain of thought reasoning:** Prompting the model to break complex problems into logical steps.
- **Multi-agent systems:** Multiple models interacting to solve complex tasks.
- **Human-in-the-loop:** Incorporating human feedback and expertise into the model training process.
- **Explainable AI (XAI):** Techniques to make the model's decision-making process transparent and interpretable.
- **Ethical AI:** Considerations and practices for ensuring responsible development and use of the model.
- **Bias mitigation:** Techniques to identify and reduce biases in the model's training data and predictions.
- **Fairness assessment:** Evaluating the model's performance across different demographic groups to ensure fairness.
- **Privacy preserving techniques:** Methods to protect sensitive data during the model training process.
- **Federated learning:** Training models on decentralized data sources without centralizing the data.
- **Differential privacy:** Adding noise to the data or the model's outputs to prevent the leakage of sensitive information.
- **Synthetic data generation:** Creating artificial data that mimics the statistical properties of real data.
- **Data augmentation:** Creating variations of the training data to increase its diversity and volume.
- **Feature engineering:** Creating new features from existing data to improve the model's performance.
- **Hyperparameter tuning:** Optimizing the model's hyperparameters to achieve the best performance.
- **Model compression:** Techniques to reduce the model's size and complexity while maintaining its performance.
- **Quantization:** Reducing the precision of the model's weights and activations to decrease memory and computational requirements.
- **Pruning:** Removing unnecessary connections or neurons in the model to reduce its size and improve efficiency.
- **Distillation:** Training a smaller model to mimic the behavior of a larger, more complex model.
- **Ensemble methods:** Combining multiple models to improve overall performance and robustness.
- **Transfer learning:** Leveraging knowledge from pre-trained models on related tasks to improve performance on the target task.
- **Fine-tuning:** Adjusting the weights of a pre-trained model on specific data to adapt it to a new task.
- **Prompt engineering:** Crafting input prompts to guide the model's behavior and elicit desired outputs.
- **Retrieval augmented generation (RAG):** Combining retrieval and generation models to leverage external knowledge bases.
- **Chain of thought reasoning:** Prompting the model to break complex problems into logical steps.
- **Multi-agent systems:** Multiple models interacting to solve complex tasks.
- **Human-in-the-loop:** Incorporating human feedback and expertise into the model training process.
- **Explainable AI (XAI):** Techniques to make the model's decision-making process transparent and interpretable.
- **Ethical AI:** Considerations and practices for ensuring responsible development and use of the model.
- **Bias mitigation:** Techniques to identify and reduce biases in the model's training data and predictions.
- **Fairness assessment:** Evaluating the model's performance across different demographic groups to ensure fairness.
- **Privacy preserving techniques:** Methods to protect sensitive data during the model training process.
- **

Figure 15: Data preprocessing pipeline in Search

Let's examine the following key steps in data preparation:

- **Data Cleaning:**
- **Data Augmentation:**
- **Feature Engineering:**

Data collection

Advanced Search models have billions of parameters that enable them to learn and generalize from data. Due to that, now, these models require extensive datasets for training to achieve complete patterns. For instance, Llama 3 was trained on 15 trillion tokens from various internet sources equivalent to 50 terabytes of data. To put this into perspective, a person reading nonstop at the rate of one word per second would take over 47,000 years to read this much text. This underscores the vast data requirements pertinent large language models to learn from different sources (e.g., web pages, scientific papers, and books).

As models grow larger, there's a trend toward enhancing training datasets with AI-generated content. This involves using existing models to create synthetic data, which can then be used to train more sophisticated models.

Figure 16: Augmenting training data with AI-generated data

Training Search models with AI-generated content has several pros and cons.

Pros:

- **Improving data diversity:** AI-generated content adds variety to existing data, thus enhancing the model's ability to generalize across different search queries and user interactions.
- **Addressing data gaps:** In scenarios where real data is scarce or insufficient, AI-generated content provides a viable alternative to ensure the model has enough examples to learn from.
- **Cost-effective:** Generating high-quality AI-generated data is often more cost-effective than collecting and annotating real-world data.

Cons:

- **Quality concerns:** The quality of synthetic data depends on the original model. Poor-quality data can lead to the spread of errors or biases.
- **Representation issues:** The synthetic data might not represent the original data well. Ensuring the synthetic data is diverse and accurate is crucial to avoid bias.
- **Real-world disconnect:** AI-generated data may not fully capture the nuances and complexities of real-world scenarios, leading to models that perform well on synthetic data but struggle with real data.

Using AI-generated data to train Search models is a rapidly evolving area of research. New techniques are constantly being developed to address the challenges and maximize the benefits of using synthetic data.

Data cleaning

Raw usage datasets from the internet are often noisy and they may contain low-quality or inappropriate content. We must clean the data carefully to avoid introducing biases, misinformation, or harmful material into the model. Short yet effect ful cleaning techniques include:

- **Deduplication:** To prevent the model from memorizing duplicate content and ensuring diverse training examples, we remove exact and near-duplicate entries.

Figure 17: Common data cleaning steps

Throughout this book, we will explore key data cleaning techniques including filtering harmful content, detecting NSFW (Not Safe For Work) language, personal data, and ensuring readability.

Data efficiency

Managing large datasets requires efficient tools and techniques for storage and retrieval. Let's look at each in detail:

Efficient storage

Storing massive amounts of data with traditional tools can be expensive and slow. Distributed storage systems such as Hadoop Distributed File System (HDFS) or cloud-based solutions like Amazon S3 or Google Cloud Storage offer scalable and cost-effective alternatives. These systems are particularly suited for managing large volumes of unstructured data. In addition, columnar storage formats such as Parquet (.pqt) and ORC (.ORC) are ideal for structured data or semi-structured data that has been converted to structured form. These formats optimized for analytical queries, offer faster data retrieval and processing capabilities.

Efficient retrieval

Training AI models requires fast access to data. Techniques like memory-mapping allows efficient data loading by accessing data directly from disk without loading the entire dataset into memory. Streaming data loading pipelines, which fetch data in batches during training, help scale to large datasets that don't fit in memory. Caching frequ ently accessed data in memory or on high-speed storage further optimizes retrieval speed, reducing bottlenecks and improving overall training efficiency.

Tokenizing

Raw text must be converted into numerical data before it can be fed into a machine learning model. Tokenization is the process of breaking down text into smaller units (tokens) such as ,ords, subwords, or characters, then assigning each token a unique numerical identifier. Modern tokenizers often employ subword tokenization algorithms, like Byte-Pair Encoding (BPE) or WordPiece, which balance vocabulary size and the ability to handle out-of-vocabulary words effectively. These tokenizers are trained on the entire dataset to ensure consistent tokenization across all data points.

Data packing

Transformer models process data in fixed-length sequences. To maximize efficiency, we pack multiple tokenized sequences into longer sequences that fill the model's context window. This minimizes wasted computation on padding tokens. Techniques like bin packing can optimize this process by strategically combining sequences of different lengths.

Efficient dataloading

Dataloaders are responsible for feeding data to the model during training. Efficient dataloaders are crucial for minimizing idle time and maximizing GPU utilization. Techniques like prefetching (loading data in advance of its need) and asynchronous data loading (overlapping data loading with model training) can significantly improve training speed.

Model development

Model development is a critical stage in building ML systems. It involves selecting the appropriate architecture, training the model, and fine-tuning pre-trained models for specific tasks. This section delves into these key aspects of model development.

Model architecture

In this step, you should talk about the architecture of the model in detail. There might be several viable architectures for different ML algorithms. For instance, effective modern models can be built using either the U-Net or DCT architectures. In an interview, it's crucial to demonstrate your understanding of the pros and cons of different architectural choices.

Since an architecture is extensive, it's helpful to analyze its specific layers and examine how the input is transformed into the output. For instance, in a convolutional neural network, you might discuss the ideal number of layers, the optimal kernel size for each convolutional layer, and the activation functions to employ.

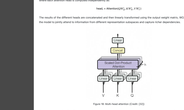

Figure 18: Simple architecture of a CNN

Transformers are a cornerstone of modern Search, especially in natural language processing and image generation. Since their introduction in 2017, they have rapidly taken over the AI community, becoming the dominant architecture for a wide range of tasks. The transformer's strength lies in its attention mechanism, which allows the model to weigh the importance of different parts of the input when making predictions. At the core of Transformers is the attention mechanism, initially introduced in the context of machine translation [17], the attention mechanism enables a model to focus on relevant parts of previous output when generating the next output. In the context of Search, it addresses the limitations of traditional sequence models such as RNNs and LSTMs by enabling the model to more effectively capture long-range dependencies and relationships between words.

Self attention, also known as scaled dot-product attention, is the most common type of attention mechanism used in modern transformers. It allows each word in a sequence to attend to all other words, including itself, to understand their relationships and context better. For instance, in the sentence "The cat sat on the mat," self-attention might assign higher weights to "cat" and "sat" when processing the word "mat" to capture the relevant spatial relationship. These scores are computed using learnable weight matrices for W_q, W_k, and W_v:

$$Q = XW_q,\ K = XW_k,\ V = XW_v$$

where, X represents the input sequence of embeddings.

The attention scores are computed by taking the dot product of the query vectors (Q), with all the key vectors, K, followed by a scaling operation and a softmax function. This can be represented as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Here, d_k is the dimension of the key vectors, and the scaling factor $\sqrt{d_k}$ is used to prevent the dot product values from becoming too large, which could lead to extremely small gradients during backpropagation. The softmax function is applied to ensure that the attention scores are normalized, summing to 1. This produces a weighted sum of the value vectors, V, where the weights are determined by the relevance of each input token as indicated by the attention scores.

Figure 19: Multi-head attention (Credit: [32])

Multi-head attention

Rather than a single attention layer, modern transformers employ multi-head attention mechanisms to enhance the model's ability to capture different types of relationships and contextual nuances. This involves dividing the input into multiple heads, each with its own learnable weight matrices for W_q, W_k, and W_v. The outputs from these heads are then concatenated and linearly transformed using the output weight matrix, W_o. This allows the model to simultaneously attend to information from different representation subspaces and capture diverse relationships.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \ldots, \text{head}_h)W_o$$

where each attention head is computed independently as:

$$\text{head}_i = \text{Attention}(QW_q^i, KW_k^i, VW_v^i)$$

The results of the different heads are concatenated and then linearly transformed using the output weight matrix, W_o. This allows the model to jointly attend to information from different representation subspaces at different positions.

Figure 20: Multi-head attention (Credit: [32])

There is no one-size-fits-all architecture for vision problems. Researchers seek to assess your understanding of different ML architectures, their strengths and weaknesses, and your ability to choose the right one based on specific requirements and constraints. In this section, we'll explore some of the prominent architectures you should be familiar with for Search.

Model training

Model training is the process of adjusting the model's parameters (weights) to produce the desired output. Key aspects to discuss during model training include:

- Loss function selection
- Training strategy
- ML objective and loss function
- Task-specific challenges and techniques

Let's explore each in more detail.

Training methodology

Each model follows a distinct training process suited to its architecture and purpose. For example, diffusion models probably involve adding noise to images and then learning to remove it, while GANs involve a competitive training dynamic between a generator and a discriminator network. Understanding these unique training loops is crucial.

Many models also undergo multi-stage training for better performance. Look, for instance, typically forms three stages: pre-training on large datasets to learn general patterns, supervised fine-tuning on curated datasets for specific tasks, and reinforcement learning to ensure outputs align with human values or intended behaviors. This approach helps models perform well across various applications.

Training an AI model is fundamentally of these training methodologies for different Search applications is crucially especially when discussing them in an interview.

Training data

Understanding the training data is essential for successful model development. The data used can vary across different Search applications, but it's important to consider factors like data quality, relevance to the task, and potential biases. For instance, models such as Common Crawl [28] might be used during the initial pre-training stage, but more specialized, carefully curated data would be preferred for fine-tuning to ensure high-quality outputs.

It's important to discuss the data collection process, including how they are sourced, why they are suitable for model training, and an estimate of their size for effective training.

Loss function

The loss function measures how closely the model's predictions align with the desired outcomes. It guides the optimization process during training, with the goal of minimizing loss over time. Common loss functions include cross-entropy for classification and pixel-wise mean squared error (MSE) for tasks like image generation. The choice of loss function depends on the specific task and the desired outcomes.

Designing a new loss function can be complex. In most cases, you'll adapt one of the widely accepted loss functions or combine several existing ones. However, understanding the trade-offs of different loss functions is important in order to fine-tune the model for optimal performance.

Task-specific challenges and mitigations

Different tasks come with their unique challenges that require specialized techniques. For example, training large video generation models demands significant computational resources and memory bandwidth to process the vast amount of data in video sequences. To tackle this, techniques such as mixed-precision training and gradient checkpointing can be employed. Other challenges might include ensuring temporal consistency in generated videos or addressing biases in the training data. For instance, in a text-to-image generation task, the model might inadvertently amplify biases present in the training data, such as gender stereotypes in occupation-related prompts. To address this, techniques like bias mitigation during data preparation or post-processing of the model's outputs may be necessary.

Figure 21: Loss computation framework

The loss function measures how closely the model's predictions align with the desired outcomes. It guides the optimization process during training, with the goal of minimizing loss over time. Common loss functions include cross-entropy for classification and pixel-wise mean squared error (MSE) for tasks like image generation. The choice of loss function depends on the specific task and the desired outcomes.

Designing a new loss function can be complex. In most cases, you'll adapt one of the widely accepted loss functions or combine several existing ones. However, understanding the trade-offs of different loss functions is important in order to fine-tune the model for optimal performance.

Task-specific challenges and mitigations

Different tasks come with their unique challenges that require specialized techniques. For example, training large video generation models demands significant computational resources and memory bandwidth to process the vast amount of data in video sequences. To tackle this, techniques such as mixed-precision training and gradient checkpointing can be employed. Other challenges might include ensuring temporal consistency in generated videos or addressing biases in the training data. For instance, in a text-to-image generation task, the model might inadvertently amplify biases present in the training data, such as gender stereotypes in occupation-related prompts. To address this, techniques like bias mitigation during data preparation or post-processing of the model's outputs may be necessary.

Figure 22: Common distributed training techniques

Training large models requires significant computational resources and memory bandwidth. To address these challenges, various techniques can be employed to distribute the training process across multiple GPUs or machines. Data parallelism involves replicating the model on each GPU and feeding different batches of data to each GPU, with gradients aggregated and synchronized across all GPUs. Model parallelism distributes the model itself across multiple GPUs, useful when the model is too large to fit into the memory of a single GPU. Pipeline parallelism divides the model into stages, with each GPU processing a different stage in a pipeline fashion. Tensor parallelism splits individual tensors across GPUs, enabling more fine-grained parallelism. Expert parallelism leverages sparsity in the model, routing different examples to different expert subnetworks. Hybrid approaches combine these techniques to maximize efficiency and scalability.

Hybrid parallelism

In practice, large-scale AI models often employ a combination of these parallelism techniques to balance computation, memory, and communication across GPUs or machines. For instance, a large language model might use data parallelism across groups of GPUs, model parallelism within each group, and pipeline parallelism between the groups. The optimal strategy depends on the specific model architecture, hardware configuration, and the trade-offs between computation speed and memory usage.

Figure 23: Common distributed training techniques

There are various methods for updating model parameters across the devices:

- **Synchronous SGD:** This ensures consistent model updates by waiting for all GPU gradients to be processed. The gradients are then averaged and used to update the model parameters across every device, maintaining model consistency. However, this approach can be slowed by waiting for the slowest parameters due to all devices. This process, conversely, as the device always work with the same model parameters, ensuring consistent updates and potentially faster convergence in homogeneous hardware setups.
- **Asynchronous SGD:** Asynchronous updating, each device works independently, updating the model parameters as soon as it finishes processing its portion of data, and the parameter server updates the model immediately upon receiving gradients from any device. This method eliminates waiting time and can lead to faster training in heterogeneous hardware setups. However, it can lead to inconsistencies and slower convergence due to model updates being based on slightly different versions of the model at any given time.

To learn more about data parallelism, refer to [29].

Model parallelism

In model parallelism, a single model is split across multiple devices, each device is responsible for computing only a portion of the model's parameters. This approach is helpful when the model is too large to fit into the memory of a single device.

There are various ways to split a model across devices:

- Pipeline parallelism (inter-layer parallelism)
- Tensor parallelism (intra-layer)

Pipeline parallelism with PP, the model layers are split across multiple devices, and computations are performed in a pipelined manner. In the forward pass, each device forwards the intermediate activations to the next device in the pipeline, while in the backward pass, it sends the gradients of these activations back to the previous device.

Figure 24: Illustrating tensor parallelism

TP is particularly useful when dealing with very large models, as it allows multiple devices to work concurrently, reducing the time spent on forward and backward passes. However, it introduces communication overhead.

Tensor parallelism (TP) is a technique where a single layer of the model is split across multiple devices. Each device handles a portion of the computation for that layer and the outputs are combined before moving to the next layer. For example, in a large matrix multiplication operation, different parts of the matrix can be processed in parallel across multiple devices. This can be done using both column-wise and row-wise tensor splits, as illustrated in the figure above.

Hybrid parallelism

In practice, large-scale AI models often employ a combination of these parallelism techniques to balance computation, memory, and communication across GPUs or machines. For instance, a large language model might use data parallelism across groups of GPUs, model parallelism within each group, and pipeline parallelism between the groups. The optimal strategy depends on the specific model architecture, hardware configuration, and the trade-offs between computation speed and memory usage.

In addition to hybrid parallelism, techniques like **ZeRO** (Zero Redundancy Optimizer) [30] from Microsoft and **FSDP** (Fully Sharded Data Parallel) from Meta offer advanced memory optimization and communication efficiency. These methods reduce redundancy in memory and computation across devices, enabling more efficient training of massive models.

The techniques discussed in this section are essential components of modern large-scale deep and ML-based training pipelines. By combining parallelism techniques such as FSDP, memory-saving methods such as gradient checkpointing, and optimizations like mixed precision training, it's possible to push the boundaries of what's feasible in terms of model size and complexity. However, it's important to strike a balance between these techniques to avoid introducing excessive overhead or complexity that outweighs the benefits. It is not the same thing as Free lunch. For those interested in a more detailed exposition of distributed training, refer to the resources mentioned in [31].

Model sampling

During inference, or at least, the test step is sampling, which involves generating new data or outputs from the trained generator model. Various sampling techniques exist, each with its unique trade-offs. For example, in GAN, techniques like truncation sampling, latent search [32], and top-k sampling [33] each have their strengths and weaknesses. Beam search, for instance, tends to produce coherent and relevant text, but it may lack diversity.

Figure 25: Sampling from text-to-text models

In an interview, it is crucial to discuss various sampling methods in depth, their pros and cons, and to choose the one that suits the system you are designing.

Evaluation

Offline evaluation

After developing a model, the next critical step is evaluation. This involves using various metrics to assess the performance of the model, ensuring it meets the desired standards before deployment. Offline evaluation is a crucial first step in this process.

Offline Evaluation

Offline evaluation is the process of assessing the performance of a model or system using pre-collected data without deploying it in a real-world environment. This evaluation typically occurs after the model has been trained but before it is deployed.

Offline evaluation offers numerous advantages, including the ability to quickly iterate on different models or to make modifications to an evaluation set, the evaluation comparison process remains consistent across the ground truth. Traditional metrics such as accuracy, precision, recall, F1-score, and mean squared error (MSE) provide a baseline for assessing model performance. These if-for common tasks but offered structured feedback, aiding in the development and refinement of models before deployment.

Task	Metrics
Classification	Precision, Recall, F1-score, Accuracy, Confusion matrix
Regression	MSE, MAE, RMSE
Ranking	Precision@k, Recall@k, MRR, mAP, nDCG

Table 3: Popular metrics to evaluate tasks

In generative models, evaluation is more complex due to the subjective nature of the output. For example, in a text-to-image generation task, human judgment is often required to assess the quality and relevance of the generated images. Metrics like Inception Score (IS) and Fréchet Inception Distance (FID) are commonly used to quantify aspects of image quality and diversity. Table 4 lists various metrics for different generative tasks.

Task	Metrics
Text Generation	Perplexity, BLEU, METEOR, ROUGE, CIDEr
Image Generation	FID, IS, SSIM, PSNR, LPIPS
Text-to-Video	FID, CLIPSIM, FVD, LPIPS, IS

Table 4: Popular metrics to generative tasks

In interviews, it's essential to discuss specific metrics for evaluating the task at hand. For example, in a text-to-image generation task, you might mention using FID to assess the quality and diversity of the generated images and CLIPSIM to evaluate how well the images align with the provided text descriptions. Various different tasks and datasets may require different evaluation metrics, and staying up-to-date with the latest advancements in evaluation methods is important for maintaining the relevance and rigor of your evaluation process.

Offline evaluation provides crucial insights into model performance before deployment. However, it's important to acknowledge that offline metrics don't always perfectly correlate with real-world performance. Factors like data distribution shifts, user behavior changes, and the emergence of new edge cases can impact the model's effectiveness in production. Therefore, while offline evaluation is a necessary and valuable step, it should be complemented by online evaluation methods to ensure the model performs optimally in the real world.

Online Evaluation

The online evaluation assesses how the model performs in production (i.e., after deployment). Different metrics aligned with business objectives are used to evaluate the model's impact.

A comprehensive online evaluation system might include various metrics. Starting an interview, you should focus on selecting the most relevant metrics for the specific task at hand. Unlike offline metrics, choosing online metrics is more subjective and often involves business considerations. For instance, in a recommendation system, you might prioritize metrics like click-through rate or dwell time to ensure the relevance of the recommended content, while monitoring user feedback provides more detailed insights into user preferences.

Metric	Description
Click Through Rate (CTR)	Percentage of users who click on a recommendation
Conversion Rate	Percentage of users who complete a desired action (e.g., purchase, subscription) after interacting with a recommendation
Dwell Time	Time spent by the model's generated content
Engagement Rate	Measure of user interaction, such as time spent, likes, or shares
Revenue Per Unit	Average revenue generated per user
Churn Rate	Percentage of users who stop using the system over a given period
User Retention	Percentage of users who continue to use the system over a specific period
User Satisfaction	Overall satisfaction of users with the recommendations, often measured through surveys or feedback

Table 5: Popular online metrics for evaluation

Table 4: Common metrics for online evaluation

Talking points

Here are some key talking points for the evaluation stage:

- **Offline metrics:** Which offline metrics best evaluate the quality and accuracy of the generative model? How do these metrics measure the diversity, realism, and coherence of generated outputs?
- **Online metrics:** Which metrics are crucial for assessing the effectiveness of the generative model in a live production environment? How do these metrics align with the business goals, such as enhancing user creativity, boosting engagement, or driving product innovation?
- **Bias:** Generative models may unintentionally reflect societal biases in relation to sensitive attributes such as gender or race. How can you evaluate the model's bias?
- **Robustness and security:** How resilient is the generative model to adversarial attacks such as intentionally misleading inputs designed to exploit model weaknesses?
- **Human evaluation:** For generative models, especially in creative fields (e.g., text generation, image synthesis), human feedback is vital. How can human reviewers complement automatic evaluation? What methods (surveys, A/B testing, expert reviews) will best assess the model's performance? How can you mitigate the effects of subjectivity among different reviewers?

Overall ML system design

The next step of the framework is to propose an overall design for the GenAI system. These systems involve more than just training a model; they require multiple pipelines and components working together seamlessly. For instance, in a chatbot, beyond the core model, other components are needed to ensure safety, for example, filtering out harmful content. Similarly, for a video generation system, additional models may be needed to upscale video resolution to the desired quality. At this stage, it's crucial to integrate all components essential for the system to function as intended. We will explore several components commonly used along with the generative model throughout this book.

Talking points

- **System components:** What are the different components of the system? What are the roles of each component—such as the core model, preprocessing, content filtering, post-processing, and any necessary upscaling or quality enhancement models?
- **Safety mechanisms:** How are safety and content moderation incorporated into the system? For instance, how does the system ensure that generated content is safe and appropriate for users? Describe components such as NSFW filters and harmful.
- **User feedback and continuous learning:** How does the system incorporate user feedback to continuously improve model performance? Discuss the feedback loop mechanisms that allow for finetuning the model. What systems are in place for retraining models with updated data to improve accuracy and relevance over time?
- **Scalability:** How does the system scale as demand increases? What cloud or hardware resources are utilized, and how is resource allocation managed efficiently? How do components such as load balancers, distributed inference, and model parallelism contribute to system scalability?
- **Security considerations:** How does the system ensure user privacy, especially when dealing with sensitive data or generating personalized content? What security protocols are implemented to protect against adversarial attacks, model tampering, or data leakage?
- **Bias:** Generative models may unintentionally reflect societal biases with respect to sensitive attributes like gender or race. Discuss strategies such as bias-detection algorithms, fairness auditing, and filtering of biased outputs. Additionally, how would you address ethical concerns if users attempt to misuse the generative model to produce harmful, biased, or inappropriate content?
- **Robustness and security:** How resilient is the generative model to adversarial attacks, such as intentionally misleading inputs designed to exploit model weaknesses? For example, can attackers generate harmful or nonsensical outputs? In production, how can we ensure that the model isn't being manipulated for malicious purposes, such as creating deepfakes, misinformation, or inappropriate content?

Deployment and monitoring

The final step is to deploy it to production and serve millions of users. Once the system is deployed, it can fail for many reasons. Monitoring refers to the task of tracking, measuring, and logging different metrics to detect system failures when they occur, so they can be fixed as quickly as possible. However, since this topic is broad and not specific to GenAI or any particular task, we won't go into detail in this book. We encourage readers to refer to [53] or the "*ML System Design Interview*" book [1] for a deeper exploration.

Summary

In this chapter, we provided an overview of GenAI and introduced a framework for approaching a GenAI system design interview. While some details are specific to GenAI, many concepts apply broadly across AI system design. We focus on aspects unique to GenAI, avoiding general topics common to all AI systems such as deployment, infrastructure, and monitoring.

Finally, not every engineer is expected to be an expert in all areas of the GenAI life cycle. Different roles and companies may emphasize various aspects, such as infrastructure, monitoring, or LLM development. This framework helps candidates to understand the expectations and adjust their answers accordingly based on the interview's focus.

Now that you understand these fundamentals, we're ready to tackle some of the most common GenAI system design interview questions.

References

[1] Machine Learning System Design Interview. https://www.aliaminian.com/books.

[2] Support Vector Machines. https://scikit-learn.org/stable/modules/svm.html.

[3] Bayes' theorem. https://en.wikipedia.org/wiki/Bayes%27_theorem.

[4] Gaussian mixture models. https://scikit-learn.org/1.5/modules/mixture.html.

[5] Hidden Markov model. https://en.wikipedia.org/wiki/Hidden_Markov_model.

[6] Boltzmann machine. https://en.wikipedia.org/wiki/Boltzmann_machine.

[7] OpenAI's ChatGPT. https://openai.com/index/chatgpt/.

[8] Economic Potential of Generative AI. https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/the-economic-potential-of-generative-ai-the-next-productivity-frontier.

[9] The Llama 3 Herd of Models. https://arxiv.org/abs/2407.21783.

[10] Flamingo: a Visual Language Model for Few-Shot Learning. https://arxiv.org/abs/2204.14198.

[11] PaLM: Scaling Language Modeling with Pathways. https://arxiv.org/abs/2204.02311.

[12] Language Models are Few-Shot Learners. https://arxiv.org/abs/2005.14165.

[13] Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding. https://arxiv.org/abs/2205.11487.

[14] PaLM2 Technical Report. https://arxiv.org/abs/2305.10403.

[15] H100 Tensor Core GPU. https://www.nvidia.com/en-us/data-center/h100/.

[16] GPT-4 training cost. www.wired.com/story/openai-ceo-sam-altman-the-age-of-giant-ai-models-is-already-over/.

[17] Scaling Laws for Neural Language Models. https://arxiv.org/abs/2001.08361.

[18] Training Compute-Optimal Large Language Models. https://arxiv.org/abs/2203.15556.

[19] Introducing OpenAI o1. https://openai.com/index/introducing-openai-o1-preview/.

[20] Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. https://arxiv.org/abs/2407.21787.

[21] ETL. https://aws.amazon.com/what-is/etl/.

[22] Tecton. https://www.tecton.ai/feature-store/.

[23] Amazon SageMaker. https://aws.amazon.com/sagemaker/.

[24] ML System Design Interview. https://www.amazon.com/gp/product/1736049127/.

[25] Comprehensive Exploration of Synthetic Data Generation: A Survey. https://arxiv.org/abs/2401.02524.

[26] HDFS Architecture Guide. https://hadoop.apache.org/docs/r1.2.1/hdfs_design.html.

[27] Amazon S3. https://aws.amazon.com/s3/.

[28] Apache Parquet. https://parquet.apache.org/.

[29] Apache ORC. https://orc.apache.org/docs/.

[30] Apache Lucene. https://lucene.apache.org/.

[31] Elasticsearch. https://www.elastic.co/elasticsearch.

[32] Attention Is All You Need. https://arxiv.org/abs/1706.03762.

[33] BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. https://arxiv.org/abs/1810.04805.

[34] An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. https://arxiv.org/abs/2010.11929.

[35] Learning Transferable Visual Models From Natural Language Supervision. https://arxiv.org/abs/2103.00020.

[36] Zero-Shot Text-to-Image Generation. https://arxiv.org/abs/2102.12092.

[37] Neural Machine Translation by Jointly Learning to Align and Translate. https://arxiv.org/abs/1409.0473.

[38] Common Crawl. https://commoncrawl.org/.

[39] Data Parallelism. https://en.wikipedia.org/wiki/Data_parallelism.

[40] Model Parallelism. https://huggingface.co/docs/transformers/v4.15.0/en/parallelism.

[41] Pipeline Parallelism. https://pytorch.org/docs/stable/distributed.pipelining.html.

[42] Mixed Precision Training. https://arxiv.org/abs/1710.03740.

[43] High-Resolution Image Synthesis with Latent Diffusion Models. https://arxiv.org/abs/2112.10752.

[44] Training Deep Nets with Sublinear Memory Cost. https://arxiv.org/abs/1604.06174.

[45] Automatic Mixed Precision. https://pytorch.org/tutorials/recipes/recipes/amp_recipe.html.

[46] Model parallelism. https://huggingface.co/docs/transformers/v4.17.0/en/parallelism.

[47] Paradigms of Parallelism. https://colossalai.org/docs/concepts/paradigms_of_parallelism/.

[48] Tensor Parallelism tutorial. https://pytorch.org/tutorials/intermediate/TP_tutorial.html.

[49] ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. https://arxiv.org/abs/1910.02054.

[50] Introducing PyTorch Fully Sharded Data Parallel (FSDP) API. https://pytorch.org/blog/introducing-pytorch-fully-sharded-data-parallel-api/.

[51] Beam search. https://en.wikipedia.org/wiki/Beam_search.

[52] Top-k sampling. https://docs.cohere.com/docs/controlling-generation-with-top-k-top-p.

[53] Model monitoring for ML in production. https://www.evidentlyai.com/ml-in-production/model-monitoring.

Partner With Us

Be an Affiliate

Become a Contributor

Company & Legal

Our Team

Newsletter

Privacy Policy

Terms of Service

Do Not Sell or Share My Personal Information

Support

hi@bytebytego.com

Report a Bug

Resources

YouTube

Visual Dev Guides

Find Jobs

Prepare for Coding Interviews

Ask Alex

Copyright ©2022-2026 ByteByteGo Inc. All rights reserved.